

 Latest updates: <https://dl.acm.org/doi/10.1145/3715752>

RESEARCH-ARTICLE

Automated Soap Opera Testing Directed by LLMs and Scenario Knowledge: Feasibility, Challenges, and Road Ahead

YANQI SU, The Australian National University, Canberra, ACT, Australia

ZHENCHANG XING, Commonwealth Scientific and Industrial Research Organisation, Canberra, ACT, Australia

CHONG WANG, Nanyang Technological University, Singapore City, Singapore

CHUNYANG CHEN, Technical University of Munich, Munich, Bayern, Germany

SHERRY (XIWEI) XU, Commonwealth Scientific and Industrial Research Organisation, Canberra, ACT, Australia

QINGHUA LU, Commonwealth Scientific and Industrial Research Organisation, Canberra, ACT, Australia

[View all](#)[Open Access Support](#) provided by:[Technical University of Munich](#)[Commonwealth Scientific and Industrial Research Organisation](#)[Nanyang Technological University](#)[The Australian National University](#)**Published:** 19 June 2025
Accepted: 14 January 2025
Received: 13 September 2024[Citation in BibTeX format](#)

Automated Soap Opera Testing Directed by LLMs and Scenario Knowledge: Feasibility, Challenges, and Road Ahead

YANQI SU, The Australian National University, Australia

ZHENCHANG XING*, CSIRO's Data61, Australia

CHONG WANG, Nanyang Technological University, Singapore

CHUNYANG CHEN, Technical University of Munich, Germany

SHERRY (XIWEI) XU, CSIRO's Data61, Australia

QINGHUA LU, CSIRO's Data61, Australia

LIMING ZHU, CSIRO's Data61 & University of New South Wales, Australia

Exploratory testing (ET) harnesses tester's knowledge, creativity, and experience to create varying tests that uncover unexpected bugs from the end-user's perspective. Although ET has proven effective in system-level testing of interactive systems, the need for manual execution has hindered large-scale adoption. In this work, we explore the feasibility, challenges and road ahead of automated scenario-based ET (a.k.a soap opera testing). We conduct a formative study, identifying key insights for effective manual soap opera testing and challenges in automating the process. We then develop a multi-agent system leveraging LLMs and a Scenario Knowledge Graph (SKG) to automate soap opera testing. The system consists of three multi-modal agents, Planner, Player, and Detector that collaborate to execute tests and identify potential bugs. Experimental results demonstrate the potential of automated soap opera testing, but there remains a significant gap compared to manual execution, especially under-explored scenario boundaries and incorrectly identified bugs. Based on the observation, we envision road ahead for the future of automated soap opera testing, focusing on three key aspects: the synergy of neural and symbolic approaches, human-AI co-learning, and the integration of soap opera testing with broader software engineering practices. These insights aim to guide and inspire the future research.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Soap Opera Testing, Large Language Models, Knowledge Graph

ACM Reference Format:

Yanqi Su, Zhenchang Xing, Chong Wang, Chunyang Chen, Sherry (Xiwei) Xu, Qinghua Lu, and Liming Zhu. 2025. Automated Soap Opera Testing Directed by LLMs and Scenario Knowledge: Feasibility, Challenges, and Road Ahead. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE035 (July 2025), 22 pages. <https://doi.org/10.1145/3715752>

1 Introduction

Exploratory testing (ET) [10, 21, 45] leverages the knowledge, creativity, and experience of testers to create varying and diverse tests on the fly to provoke and identify unexpected bugs. Unlike scripted

*Corresponding Author

Authors' Contact Information: Yanqi Su, Yanqi.Su@anu.edu.au, The Australian National University, Australia; Zhenchang Xing, Zhenchang.Xing@data61.csiro.au, CSIRO's Data61, Australia; Chong Wang, chong.wang@ntu.edu.sg, Nanyang Technological University, Singapore; Chunyang Chen, chun-yang.chen@tum.de, Technical University of Munich, Germany; Sherry (Xiwei) Xu, xiwei.xu@data61.csiro.au, CSIRO's Data61, Australia; Qinghua Lu, qinghua.lu@data61.csiro.au, CSIRO's Data61, Australia; Liming Zhu, liming.zhu@data61.csiro.au, CSIRO's Data61 & University of New South Wales, Australia.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2025/7-ARTFSE035

<https://doi.org/10.1145/3715752>

testing, where test cases are executed in a mechanical and repetitive way, ET allows for open-ended results [10, 21, 45]. Some outcomes may be expected and predicted; others may not. Testers engage by configuring, operating, observing, and evaluating the system, critically examining outcomes, and reporting any potential bugs. Although scripted testing has received much attention in the research community, ET is widely practiced and highly valued in the software industry [17–19, 32]. Some scientific studies [17–19] show ET is an effective and efficient testing approach for system-level testing of interactive systems through the GUI and from the end user’s viewpoint, and can find functional, usability, performance and security bugs [32, 36].

Cem Kaner and others [10, 21, 45] have established concrete principles for ET, emphasizing that *not scripting does not mean not preparing* [21]. Exploratory testers need to gather, analyze, and study extensive information about the application domain, software products, and users. ET encompasses diverse approaches, enabling testers to select the most suitable one based on their goals or needs, such as feedback-based ET or strategy-based ET. Scenario-based ET, also known as soap opera testing [8], involves designing complex test scenarios to provoke failures and recognize them. Recent work [36–38] has constructed a system knowledge graph (SYSKG) from bug reports, modeling user tasks, failures, and system interactions to support soap opera testing. These studies generate soap opera tests by combining relevant scenarios from SYSKG. Experimental results [36–38] have shown that these tests help testers learn the product, simulate expert users, and uncover hidden failures. However, the need for manual execution has limited the large-scale adoption of soap opera testing. Leveraging the capabilities of large language models (LLMs) in natural language and vision understanding, this work explores the feasibility, challenges, and road ahead of automated soap opera testing guided by LLMs and scenario knowledge.

We conduct a formative study to investigate how humans effectively perform soap opera testing and to identify the challenges in automating this process. Eight professionals specializing in soap opera testing participated in the study. First, a user study tasks each participant with conducting soap opera testing using provided soap opera tests, aiming to uncover as many bugs as possible. Following this, we conduct interviews that reference both the bug discovery process and the participants’ prior experience to identify the insights and challenges. Two insights emerge for effective soap opera testing: (1) *discovering unexpected behaviors while exploring and observing*, and (2) *expanding exploration boundaries through creative thinking*. This means that testers need to remain observant at all stages of the soap opera test execution, continuously identifying any unexpected behaviors to uncover potential bugs. In addition, while ensuring the soap opera test is completed, testers need to engage in creative thinking to explore how different inputs affect the system and whether various side tasks along the soap opera test might trigger bugs. It’s similar to a role-playing game where, alongside completing the main quest, players can unlock side quests.

The challenges of automated soap opera testing are soap opera test auto-play and real-time bug auto-detection. Soap opera tests, written in natural language (NL), often suffer from quality issues such as missing UI instructions and outdated UI descriptions, complicating the automation process. Additionally, effective bug detection requires the system to understand both the executed instructions and the resulting UI changes in real-time, necessitating multi-modal understanding. Addressing these challenges is crucial for the effectiveness of automated soap opera testing. To explore feasibility, we developed a multi-agent system inspired by two insights, leveraging the advanced understanding capabilities of multi-modal LLMs in both NL and vision, combined with scenario knowledge to address identified challenges. Specifically, this system consists of three multi-turn-dialogue agents, namely Planner, Player, and Detector, powered by multi-modal LLMs and a scenario knowledge graph (SKG). Given a soap opera test (a list of system operation steps) and the initial GUI status, three agents collaborate to perform the testing for identifying any potential bugs. To evaluate the feasibility of automated soap opera testing, we tested 4 apps, each with 10 soap

opera tests, resulting in the submission of 44 bug reports. Of these, 3 bugs were fixed, 26 confirmed, 2 assigned priority and severity, 2 reopen, and 3 marked as duplicate. These results demonstrate the potential of automated soap opera testing, but there remains a significant gap compared to manual execution, such as under-explored scenario boundaries and incorrectly identified bugs (false positives). To better understand the high false positive rate, we analyzed the incorrectly identified bugs and identified four main causes: LLM hallucinations, screenshot-based GUI presentation, incorrect GUI element location, and unreasonable or unnecessary enhancement suggestions.

By the preliminary exploration of automated soap opera testing, we summarize key observations and outline the road ahead for future research. First, the synergy of neural and symbolic approaches is essential for effective soap opera testing. By integrating app-specific knowledge at various aspects, LLMs can enhance the understanding of software under test, thereby broadening and deepening exploration. Second, while humans provide valuable insights for AI in automated soap opera testing, AI's behavior can also inspire human creativity, uncovering bugs. As we transition from manual to automated testing, a Human-AI collaborative approach could serve as an effective middle ground. Lastly, rather than stopping after discovering a single bug, soap opera testing excels at exploring further, uncovering related issues and identifying different manifestations of the same root cause. Therefore, combining soap opera testing with broader software engineering knowledge, such as code analysis for root cause identification, is a logical next step. Our contributions are:

- We conduct a formative study to identify the insights of effective manual soap opera testing and the challenges of automated soap opera testing.
- We design a multi-agent system based on LLMs and scenario knowledge for the preliminary feasibility exploration of automated soap opera testing.
- Through extensive experimental analysis, we find that automated soap opera testing directed by LLMs and scenario knowledge is promising, but there is still a significant gap compared to manual soap opera testing.
- We outline the road ahead for automated soap opera testing, offering guidance for future research.

2 Formative Study

We conduct a formative study to investigate how humans effectively perform soap opera testing and identify the challenges in automating this process, as follows:

RQ1: How effective is manual soap opera testing?

RQ2: How can manual soap opera testing be conducted effectively?

RQ3: What are the challenges in automating soap opera testing?

2.1 Study Setup

We recruited eight professionals specializing in soap opera testing as participants, with four from industry and four from academia. Each participant has a minimum of 2 years of experience in soap opera testing for large interactive systems with GUI, such as desktop or mobile apps. Notably, four participants have experience in GUI-based automated Android testing. First, a user study tasks each participant with conducting soap opera testing to uncover bugs. Then, we conduct interviews based on the bug discovery process and the participants' prior experience, focusing on: (a) insights for effectively doing soap opera testing, and (b) challenges in automating this process.

2.1.1 User Study. Each participant is tasked with conducting soap opera testing using five predefined soap opera tests, aiming to uncover as many bugs as possible within 3 hours.

Dataset. Four open-source Android apps were selected as our software under test (SUT) from different categories: [Firefox](#), a fast and privacy-focused browser; [WordPress](#), a website builder for creating posts and tracking analytics; [AntennaPod](#), a podcast manager and player; and [Amaze](#),

a material design file manager. These apps were selected based on the 8 candidate apps from a recent study [47] on Android functional bugs, taking into account the scale of issues and active developer engagement. All apps include over 1 million downloads on Google Play (popularity), a public issue tracking system (traceability), and approximately ten years of development history (trustworthiness). For each app, we construct a dataset of 10 soap opera tests covering diverse features, such as login, bookmarks, privacy, browser engine, and translation. Each test consists of a series of system operation steps sourced from various origins: user manuals, issues, pull requests and bug reports. The average number of steps per test is 5.9 for Firefox, 5.0 for WordPress, 4.7 for AntennaPod, and 5.0 for Amaze. To ensure continuity in testing, each participant is assigned 5 soap opera tests from a app. Two participants, who had previously used Firefox, are assigned this app. The remaining participants, who had no prior experience with the apps, are randomly assigned to the other apps. This arrangement helps unveil the insights and challenges of soap opera testing when the testers is familiar or unfamiliar with the SUT.

2.1.2 Interviews. We conduct interviews with participants based on the bug discovery process from user study and their prior experience. This provides a fresh and concrete foundation for interviews, which aims at addressing the two research questions. Then, we use thematic analysis [6] to summarize and synthesize the content of the interviews, leading to the final conclusions.

First, *How can manual soap opera testing be conducted effectively?* Participants are asked to recount their bug discovery process in the user study, including how these bugs were triggered and identified through soap opera testing. Participants may back up their claims with prior testing experience. Second, *What are the challenges in automating soap opera testing?* Participants share their insights, referring to specific cases in the user study and also drawing on their general experiences with both soap opera testing and GUI-based automated testing where applicable. The discussion centers on the challenges associated with triggering and identifying bugs within the context of automation.

2.2 RQ1: Effectiveness of Manual Soap Opera Testing

The eight participants found 11, 16, 5, 9, 8, 8, 9 and 14 bugs across Firefox, WordPress, AntennaPod, and Amaze respectively. The status of identified bugs is 11 bugs fixed, 37 bugs confirmed, 2 bugs assigned priority and severity, 5 bugs assigned assignees, 10 bugs as wontfix, 5 bugs as dup. As shown in Table 2, these bugs span a diverse range of categories, including UI, functionality, performance, crash issues. We also received numerous acknowledgments from the development teams, such as: “Thank you especially for submitting so many detailed issues” and some bugs (e.g., [Issue7357](#)) also sparked discussions among developers.

2.3 RQ2: Insights from Manual Soap Opera Testing

Through the formative study, we identify two key insights for effectively conducting soap opera testing: discovering unexpected behaviors while exploring and observing, and expanding the boundaries of exploration by creative thinking. These two insights echo the key findings in [45]: *An exploratory tester who uses a scenario as a guide will often pursue interesting alternative inputs or pursue some potential side effect that is not included in the script.* – James A. WHITTAKER.

2.3.1 Discover Unexpected Behaviors While Exploring and Observing. All participants mention that unlike scripted testing, where bugs are detected based on predefined oracles after executing test scripts, soap opera testing enables the detection of unexpected bugs during the execution of intermediate steps, even beyond the known oracles. An example from Participant 2 ($P_{2\text{Firefox}}$) for Firefox on [Find in Page function](#) from the user manual illustrates this practice well. When executing the middle step (*Enter your search term ...*) of this soap opera test, $P_{2\text{Firefox}}$ discovered that the “Find in Page” feature incorrectly starts from ‘4/15’ instead of ‘1/15’ when the page is at

the top, as reported in [Bug1913291](#). $P2_{Firefox}$ then proceeded to the next step (*Navigate through search results using the up or down ...*). $P2_{Firefox}$ found that “Find in page” inconsistently fails to highlight certain search results, leading to confusion and difficulty in locating terms ([Bug1913295](#)). Based on this confusing behavior, $P2_{Firefox}$ scrolled the screen to locate the highlighted search results, discovering that scrolling while searching triggers the selection tool panel in “Find in Page” ([Bug1913304](#)). Additionally, navigating through search results caused a blue box to appear around buttons or links ([Bug1913307](#)). Furthermore, tapping on the highlighted search result or a blank area caused the highlight to disappear ([Bug1913299](#)). An error displaying “0/0” occurred when rapidly tapping the “Up/Down” buttons, but this issue had already been reported by others. However, since the steps to reproduce this bug have changed, we submitted a comment ([Bug1807147#Comment11](#)) to update based on the new observations made by $P2_{Firefox}$. After reporting these bugs, 4 have already been confirmed. For details, please refer to the bug links mentioned above.

This example also reveals another significant issue. [Bug1807147](#) was reported two years ago, but related issues in the “Find in Page” feature were missed. Developers patched only the reported bug. Had exploratory testing (ET) been conducted thoroughly, other issues could have been identified, leading to a more systematic resolution. This encourages deeper investigation of root causes, shifting from isolated bug fixes to comprehensive problem-solving.

2.3.2 Expand the Boundaries of Exploration by Creative Thinking. In addition to staying observant for any unexpected behaviors during the soap opera tests, participants also use their creativity and experience to engage in creative thinking. This allows them to generate various inputs and branches to expand the boundaries of exploration in soap opera testing. This is similar to exploring side quests in a role-play game while the player completes the main mission [36].

Input Exploration. $P1_{Firefox}$, $P2_{Firefox}$, $P7_{Amaze}$ and $P8_{Amaze}$ emphasize the importance of diverse input exploration in triggering bugs during soap opera testing, which is guided by the current scenario, experience and known bug-finding techniques to create representative inputs. For instance, $P2_{Firefox}$ encountered a scenario requiring a website input while executing a soap opera test. Instead of entering the correct website, $P2_{Firefox}$ intentionally misspelled it, which triggered the issue “Entering a misspelled website causes the loading to stall, and refreshing reverts to the previously visited page” ([Bug1913318](#)). When the soap opera test involved adding a search engine, $P2_{Firefox}$ attempted to add a search engine with the same name and details as an existing one. This led to the discovery of a confirmed bug, “Adding a search engine allows duplicate entries” ([Bug1913414](#)). When $P1_{Firefox}$ needed to edit a saved login, $P1_{Firefox}$ utilized the boundary value analysis technique by entering an extremely long username and password. This triggered the confirmed bug “Long usernames and passwords overlap with the ‘Clear’ button...” ([Bug1912207](#)).

Branch Exploration. All participants noted that while ensuring the soap opera test is executed, performing random or logically driven side tasks can be beneficial for triggering bugs.

Random Branch Exploration Triggering bugs. After $P5_{AntennaPod}$ completed the soap opera test and discovered a bug ([Issue7349](#): “After removing a podcast, its episode stays in the miniplayer”), $P5_{AntennaPod}$ randomly clicked a button on the current page, triggering a crash ([Issue7350](#)). Similarly, when $P6_{AntennaPod}$ was executing a soap opera test following the steps in [Issue5822](#), after performing “Remove Podcast”, a bug was triggered where “The removed podcast still appears under ‘Get surprised’ after confirming removal”, reported in [Issue7370](#) and subsequently fixed. After discovering this bug, $P6_{AntennaPod}$ became curious about what would happen after clicking on the removed podcast under “Get surprised”, which triggered another crash ([Issue7371](#)).

Logically Driven Side Tasks Triggering Bugs. For a soap opera test on translation, after clicking “Translate” to translate the entire page, $P2_{Firefox}$ noticed that the translation process took unusually long. To cancel the process, $P2_{Firefox}$ spontaneously clicked the “Not now” button, but fails to

cancel translation (Bug1913602). $P2_{Firefox}$ then observed that “The page translation occurs in stages, with an initial partial translation followed by a delayed translation of the remaining content” (Bug1913606). Based on this observation, $P2_{Firefox}$ attempted to refresh the page to complete the partial translation but discovered another issue: “Refreshing a translated webpage reverts it to the original language...” (Bug1915093). After exploring these branches, $P2_{Firefox}$ resumed the main task of soap opera test by tapping the ‘Reader view’ icon to return the webpage to normal mode. This revealed “Switching between Reader view and normal mode causes the page to revert to the original language...” (Bug1913604). $P2_{Firefox}$ also noticed that “The ‘Translation’ icon color on the address bar changes inconsistently” (Bug1913605). One has been fixed, while the rest were confirmed.

$P1_{Firefox}$ used equivalence class to trigger bugs. After discovering Bug1912207 (“Long usernames...overlap with the ‘Clear’...when editing saved logins”), $P1_{Firefox}$ realized that ‘Add password’ page also has input fields, which might exhibit the same issue. $P1_{Firefox}$ then navigated to the ‘Add password’ page and inputted long usernames and passwords, which led to Bug1912199: “Input Field Overlap with ‘Clear’ Button in ‘Add Password’”. One bug has been fixed, and another confirmed.

Two key insights were identified for effectively conducting soap opera testing: discovering unexpected behaviors while exploring and observing, and expanding the boundaries of exploration by creative thinking. Testers need to remain observant at all stages of the soap opera test execution, continuously identifying any unexpected behaviors. Testers also utilize their creativity and experience to generate diverse inputs and explore different branches, triggering unexpected behaviors. These insights are further validated by [1, 3, 10, 21, 29, 45] and supported by [36–38].

2.4 RQ3: Challenges in Automated Soap Opera Testing

Automated soap opera testing requires not only automatically executing the soap opera tests (Soap Opera Test Auto-play) but also identifying unexpected bugs that may arise at any point during execution (Real-Time Bug Auto-detection).

2.4.1 Soap Opera Test Auto-play. All participants identified a key challenge in mapping NL steps in soap opera tests to the corresponding UI instructions for automated manipulation.

Missing UI Instructions. Missing UI instructions refers to situations where some necessary UI instructions are skipped between two steps, or where a single step in the soap opera test requires multiple UI instructions to be executed. Six participants identified this as a challenge. Even user manuals can contain instances of missing UI instructions. For example, when $P2_{Firefox}$ executed a soap opera test from the user manual [To move a bookmark to a new folder](#), the move operation failed despite following all the steps. Initially, $P2_{Firefox}$ assumed this was a bug, but after expanding the boundaries of exploration, $P2_{Firefox}$ discovered that the failure was due to a missing step in the user manual. Specifically, a step states, “Tap the ‘Back’ arrow ... a few times until you return to browsing the web”. During this process of tapping ‘Back’ a few times, the crucial UI instruction (“Tap the ‘Save’ icon to save the change”) is missing, which is necessary to complete the move operation successfully. This is an example where automated execution could lead to a false positive, resulting in the reporting of a non-existent bug. In a soap opera test from Bug1812113, the step “Open recently closed tabs” left $P1_{Firefox}$ unsure how to proceed, even with prior Firefox experience. Searching [Bugzilla](#), $P1_{Firefox}$ found detailed instructions in Bug1881509: “Open Tabs Tray”, “Tap on the three dots menu”, and “Select ‘Recently closed tabs’”. Without sufficient execution details, automated testing may halt at such steps.

Outdated UI Description. Outdated UI descriptions occur when soap opera test steps no longer match the app’s current interface due to updates. For example, “Logins and passwords” is now “Passwords”, and “Saved logins” is now “Saved passwords”. Similarly, “Navigate to History from

Settings” is now under the “Three-dot” menu. Five participants mentioned that these discrepancies hinder automated execution.

2.4.2 Real-Time Bug Auto-detection. All participants agreed that effective automated real-time bug detection requires both image recognition and NL understanding. Specifically, automated soap opera testing needs to comprehend the UI instructions being executed and the corresponding changes triggered in the UI. From the formative study, bugs were categorized into two types: inner-page, identified within a single UI (e.g., long usernames overlapping with the ‘Clear’ button), and inter-page, discovered by observing changes across multiple UI pages and actions that triggered these changes (e.g., [Bug1913604](#), where switching modes causes the page to lose translation).

The success of automated soap opera testing depends on two key challenges: ensuring accurate test auto-play and achieving real-time bug detection. Variability in test steps, such as missing or outdated UI instructions, complicates automation and can cause failures. Effective bug auto-detection also requires real-time understanding of both the executed instructions and UI changes.

3 Preliminary Exploration of Automated Soap Opera Testing

To explore the feasibility of automated soap opera testing, we developed a multi-agent system inspired by two key insights uncovered in the formative study, leveraging the superior understanding capabilities of multi-modal LLMs in both NL and vision, combined with scenario knowledge, to address the identified challenges. Specifically, to discover unexpected behaviors while exploring and observing, our system continuously detects anomalies by analyzing the current GUI status after executing each UI instruction, supported by oracles derived from both scenario knowledge and LLM generation. To expand the boundaries of exploration by creative thinking, we leverage scenario knowledge to provide varied, edge-case inputs (e.g., extremely long URLs) and a diverse set of representative inputs, such as different website URLs (e.g., [wikipedia.org](#), [https://1.0.0.1/ID.7](#)). Scenario knowledge also supports alternative execution paths based on the current GUI status. Additionally, we intentionally exploit LLMs’ error localization of GUI elements to introduce unexpected UI instructions, uncovering bugs through these “out-of-line” execution paths.

3.1 Automated Multi-agent Workflow

As shown in Figure 1, the system consists of three *multi-turn-dialogue* agents, namely *Planner*, *Player*, and *Detector*, powered by multi-modal LLMs and a scenario knowledge graph (SKG). Given a soap opera test (a list of system operation steps) and the initial GUI status (a screenshot), the three agents collaborate to perform the testing as follows:

Planner: Soap opera tests often face issues like missing steps or outdated UI descriptions, making them unsuitable for direct use. To address this, the Planner agent generate an actionable plan (detailed sub-steps) by utilizing LLMs based on the test description, the GUI status and step knowledge from SKG. As illustrated in Figure 1, based on the given test and the current GUI status (Screenshot 1), Planner identifies that the step “Close a tab” needs to be executed. Planner then generates a plan for it based on the retrieved steps in SKG, which includes four sub-steps, e.g., “Click on the tabs tray”, “Locate...”, which are more detailed and suitable for the current GUI status.

Player: From the generated plan, the Player agent processes one sub-step at a time, translating it into a structured UI instruction based on the LLMs’ understanding of both the sub-step and the current GUI status. Player then executes the instruction and inspects the resulting GUI status by taking a screenshot. For example, for the sub-step “Click on the tabs tray” and the current GUI status (Screenshot 2) in Figure 1, Player translates the sub-step into a structured instruction with a

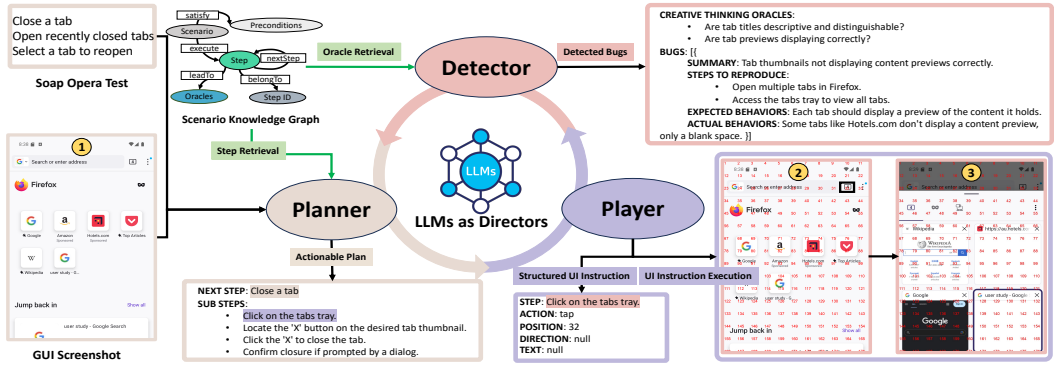


Fig. 1. Approach Overview of Automated Soap Opera Testing

“tap” action targeting element number “32” (indicating the position to tap). This instruction is then executed, advancing the GUI to the next status (Screenshot 3).

Detector: The Detector agent is employed after each Player execution to identify potential unexpected bugs that may occur at any point. Detector first retrieves relevant oracle knowledge from SKG based on the executed UI instruction. Then, using the GUI statuses before and after the Player’s execution, along with the executed sub-step and retrieved oracles, LLMs further do creative thinking to generate more potential oracles. Afterward, Detector assesses whether the subsequent GUI status violates any of these oracles, both retrieved and generated. If oracle violations are detected, bugs are identified. For example, Figure 1 illustrates a detected bug “Tab thumbnails not displaying content previews correctly” after executing the sub-step “Click on the tabs tray”. After reporting in [Bug1912739](#), this bug was confirmed. Notably, the retrieved oracles from the SKG included “Thumbnails should be displayed properly ...”, providing support for bug detection.

Loop Iteration. The above process iterates until Planner completes all steps in the given soap opera test, with Player executing instruction and Detector performing bug detection for each sub-step. To ensure the agents remain aware of the complete testing context, they are designed to reserve the previous query-response records (e.g., for Planner, query is the test description and GUI status, response is the generated plan). These records are used to generate responses in subsequent turns. To satisfy the requirement of analyzing test description and GUI simultaneously, the LLMs employed in the three agents are *multi-modal models*, such as [GPT-4o](#).

3.2 Application-Specific Knowledge Augmentation

While LLMs excel in understanding NL and vision, they lack application-specific knowledge, such as operation steps and oracles, needed for effective test auto-play and bug auto-detection. Our formative study shows participants often rely on the software’s repository for UI operations and bug identification. Thus, we construct a scenario knowledge graph (SKG) with operation steps and oracles from crowd-sourced sources like bug reports and issues, using a retrieval-augmented generation (RAG) [23] to provide this knowledge to Planner and Detector.

3.2.1 Scenario Knowledge Graph. SKG is constructed by adapting SYSKG pipeline [38]. As shown in Figure 1, SKG consists of scenarios, with each entry containing the following key information extracted from various sections of a bug report: **Summary** as a high-level description of the scenario, extracted from the “title” section; **Preconditions** as descriptions indicating the prerequisites required to execute steps, extracted from the “prerequisites” section; **Steps** as normalized step-by-step descriptions of the scenario, extracted from the “S2Rs” section; **Oracles** as descriptions of the expected behaviors, extracted from the “EBs” section. The steps and oracles serve as the primary

sources for the KG-based RAG in our Planner and Detector, while the summary and preconditions provide essential context to enable more accurate knowledge retrieval. Note that “normalized” for steps means they are deduplicated by clustering descriptions with the same meaning. Each normalized step is assigned a unique ID and includes a set of alternative descriptions, along with a corresponding set of associated oracles. (cf. the construction of SYSKG [38]). For example, a normalized step with ID ‘8’ in Firefox has 47 alternative descriptions, including “Go to Tabs Tray” and “Open tab tray”, and it has 19 corresponding oracles, such as “Tab thumbnails not displaying content previews correctly” and “Tooltip should display explanatory text”. SKG construction is a one-time, cost-effective process using LLMs (e.g., [gpt-4o-mini](#)) for extraction and an embedding model (e.g., [text-embedding-3-large](#)) for embedding. The resulting SKG costs approximately \$4.50 for Firefox (7,947 bug reports), \$8.80 for WordPress (21,010 issues/PRs), \$2.40 for AntennaPod (7,317 issues/PRs), and \$1.50 for Amaze (4,283 issues/PRs).

3.2.2 KG-Based Retrieval-Augmented Generation. To leverage the crowd-sourced knowledge about the SUT, this system integrates the SKG and employs RAG. For efficient retrieval, we transform the SKG into vector representations using chunking and embedding process. As the SKG consists of nodes and relations, we convert it into structured text, where each node’s relations are represented as attributes. For example, each step node is converted into structured text with two key properties: oracles and step ID. This structured text is then split into chunks for vectorization, with a fixed token chunk size and overlap to ensure context continuity and minimize information loss at chunk boundaries. We utilize an embedding model to convert chunks into vector representations, capturing the rich textual information from the steps and oracles. The vectorized SKG enables the agent to search the KG using both keyword-based and semantic queries. Based on the retrieved results, the agent can generate outputs that are augmented with knowledge from the SKG. We currently use the [file-search](#) interface to implement this process, where RAG is invoked by including a file search instruction in the query. The file search interface is set up using default hyper-parameter settings, such as an 800-token chunk size and a 256-dimensional embedding size.

3.3 Planner

Planner is a multi-modal agent responsible for identifying the next step to be executed and generate the actionable plan based on the current GUI status and the test description.

3.3.1 Agent Initialization. As the LLMs are originally designed for general purposes, we need to specialize it for the specific role of plan generation for test steps. This specialization is achieved by using a system-level role-play prompt, as illustrated in Figure 2. The prompt consists of three main components: *Role Assignment*, which designates the LLM as a test scenario execution planner; *Task Description*, which outlines the step-by-step instructions (four steps) to be followed, based on the chain-of-thoughts (CoTs) idea [43]; and *Output Guidelines*, which directs the LLM to respond in JSON format, including fields such as “NEXT STEP” and “SUB STEPS”, each with a brief description. To incorporate crowd-sourced knowledge about the SUT when generating an actionable plan, Planner utilizes the KG-based RAG described in Section 3.2.2. Specifically, a tailored instruction is included in the *Task Description* to prompt the agent to invoke the file search interface for RAG based on the context. The retrieved step knowledge is then used to enhance the subsequent plan generation.

Role Assignment
You are a test scenario execution planner.

Task Description
Based on the given test scenario and the current GUI status (the given screenshot),
a. observe the current GUI;
b. identify the step to be executed;
c. search the file to get all relevant test scenarios, which provide feasible sub-steps for executing the step;
d. generate feasible sub-steps to be executed based on the current GUI and relevant test scenarios. *Instruction for RAG Invocation*

Output Guidelines
Please return the output in the JSON format:
• **NEXT STEP:** Represents the step that needs to be executed next. If there are no more steps, this should be an empty string.
• **SUB STEPS:** A list of sub-steps related to the current step. If no sub-steps are left, this should be an empty list.

Fig. 2. Role-Play Prompt for Planner Initialization

Additionally, Planner maintains a test-specific dialogue session that stores previous query-response records to preserve the complete testing context.

3.3.2 Plan Generation. Given a GUI status, Planner generates the corresponding plan based on the provided test and previous query-response records in the dialogue session. Specifically, a query is constructed using the template: “Steps: [Steps]\nGUI Status: [GUI Status]”, where the placeholders are filled with the given test and GUI status. After submitting the query to the agent, a response in the JSON format defined in *Output Guidelines* is returned, from which sub-steps can be extracted to form the plan. The record of the query and response from this interaction is appended to the dialogue session for use in subsequent turns of plan generation.

3.4 Player

After receiving the actionable plan from Planner, Player translates the sub-step into a structured UI instruction for the current GUI status and then executes it.

3.4.1 Agent Initialization. Similar to Planner, Player is initialized with a three-component (i.e., *Role Assignment*, *Task Description*, and *Output Guidelines*) role-play prompt, as shown in Figure 3, and maintains a dialogue session to store previous query-response records. Additionally, Player integrates a command-line tool (e.g., ADB) to execute UI instructions. We provide nine common standard actions for UI instructions: *tap*, *long-tap*, *double-tap*, *input*, *scroll*, *home*, *enter*, *landscape*, and *portrait*. Specifically, *home* simulates pressing the home button, *enter* simulates pressing the Enter key, *landscape* rotates the device to landscape mode, and *portrait* switches it back to portrait mode. Less common actions, like pinch or multi-hand gestures, are excluded. Some actions require arguments: *tap*, *long-tap*, and *double-tap* have a *position* argument; *input* requires *position* and *text*; *scroll* takes a *direction* argument. Other actions, such as *home*, can be executed without arguments.

3.4.2 UI Instruction Translation. A UI instruction consists of two key components: *Action*, which specifies the type of instruction, and *Arguments*, which indicates the required action arguments. Given the plan and the current GUI status, Player constructs a LLM query using the template “Plan: [Plan]\nGUI Status: [GUI Status]”, where the placeholders are populated with the relevant information. The output is a JSON response following the format defined in the *Output Guidelines*, from which a structured UI instruction is generated. The query and response from this interaction are then appended to the dialogue session for use in subsequent turns of UI instruction translation.

Before inserting the GUI screenshot into the [GUI status] placeholder in the query template, we label it with numbered elements to support actions that require a *position* argument. The reasoning is as follows: A straightforward approach to represent UI positions for instructions is by directly using x-y coordinates. However, it is impractical for LLMs to predict precise widget coordinates. To address this, we divide the UI into fixed-size elements (e.g., 100px × 100px grids) and number them sequentially from left to right and top to bottom. Each element’s numeric label is overlaid on the screenshot, allowing LLMs to reference the numeric labels to specify the UI position for an instruction. For example, GUI Screenshot 2 in Figure 1 shows the labeled version of GUI Screenshot 1, where the label “32” indicates the UI element corresponding to “tabs tray”. The grid creation process is fully automated. The grid size is determined to ensure that each interactive UI element is covered by at least one grid number while preventing overlap between grid numbers.

Role Assignment

You are a helpful element locator capable of identifying the location of elements within a GUI and the actions to be executed on them, facilitating operations based on specified steps.

Task Description

Please identify which element to operate and which action to use based on the given step and specific GUI.

- If scrolling the page results in no change, it means the page cannot be scrolled further or reach the end. Please try other actions or change the scroll direction.
- If multiple attempts to operate an element fail, please try operating the element using other nearby coordinates.

Output Guidelines

Please return the output in the JSON format:

- ACTION:** Specifies the type of action to perform.
- POSITION:** A number corresponding to the numerical identifier of the element in the GUI.
- DIRECTION:** up, down, left or right.
- TEXT:** Specifies the input into the element if needed.

Fig. 3. Role-Play Prompt for Player Initialization

3.4.3 UI Instruction Execution. After generating a structured UI instruction, Player executes it using a command-line UI operation tool such as ADB [13]. For actions that require a *position* argument, the exact UI coordinates are calculated and derived from the numeric label. As shown in Figure 1, after executing the instruction “[tap] position=32”, the GUI transitions from Screenshot 2 to Screenshot 3. Player captures a new screenshot to record the updated GUI status.

3.5 Detector

To promptly identify unexpected behaviors, Detector retrieves relevant oracles from the SKG after each UI instruction execution and applies creative thinking to detect potential bugs.

3.5.1 Agent Initialization. Similar to Planner and Player, Detector is initialized with a three-component (i.e., *Role Assignment*, *Task Description*, and *Output Guidelines*) role-play prompt, shown in Figure 4, and maintains a dialogue session to store previous query-response records.

3.5.2 Oracle Retrieval. We retrieve relevant oracle knowledge from SKG to assist in bug detection as follows. First, we extract steps similar to the executed UI instruction from the SKG using the KG-based RAG method described in Section 3.2.2. As shown in Figure 5, the prompt used to invoke the RAG contains four main components: *Role Assignment*, *Task Description*, *UI Instruction*, and *Output Guidelines*. Using this prompt, steps similar to the executed UI instruction are returned by their unique IDs. Based on the retrieved steps, all associated oracles are fetched from the SKG. For example, for the UI instruction corresponding to “Click on the tabs tray” in Figure 1, the retrieval returns “Go to Tabs Tray” and “Open tab trays” as similar steps, along with several associated oracles such as “Tooltip should display explanatory text” and “Thumbnails should be displayed properly in the Tabs Tray”.

3.5.3 Bug Detection. Given the executed UI instruction and the updated GUI status, Detector performs bug detection based on the retrieved oracles and previous query-response records in the dialogue session. Specifically, a query is constructed using the template: “UI Instruction: [UI Instruction]\nGUI Status: [GUI Status]\nRetrieved Oracles: [Retrieved Oracles]”, where the placeholders are filled with the given information. After submitting the query to the agent, a response is returned in the JSON format by the *Output Guidelines*, from which bugs, including summary, S2Rs, EBs and OBs, can be extracted. The record of the query and response from this interaction is appended to the dialogue session for use in subsequent turns of bug detection.

4 Experiment

To evaluate the effectiveness of automated soap opera testing and its ability to address the challenges identified in the formative study, we assessed it as follows:

RQ1: How effective is automated testing in real-time bug auto-detection?

RQ2: How effective is automated testing in soap opera test auto-play?

RQ3: Is application-specific knowledge useful for automated testing?

RQ4: How generalizable is automated testing to a broader set of applications?

Role Assignment

You are an identifier capable of identifying bugs given the steps and their corresponding GUIs. Please use creative thinking to find any bugs.

Task Description

- generate creative thinking oracles based on the scenario context and retrieved oracles
- conduct an overall inspection of the GUI to ensure there are no obvious visual issues.
- separately check each component to ensure its layout is reasonable.
- check for bugs by analyzing the sequences of images and their corresponding operations.
- Please return the bugs that are actually observed, do not speculate.
- Note that the GUI is the state after executing the given step.
- Note that there are oracles to identify any bugs, but please do not restrict to these oracles.

Output Guidelines

Please return the output in the JSON format:

- BUGS:
 - SUMMARY: the summary of bug.
 - STEPS_TO_REPRODUCE: steps to reproduce the bug
 - EXPECTED_BEHAVIORS: expected behaviors
 - ACTUAL_BEHAVIORS: actual behaviors

Fig. 4. Role-Play Prompt for Detector Initialization

Role Assignment

You are a step identifier capable of identifying the step ID for the given step.

Task Description

- Search the file to get the ID of steps with the same UI operation as the given step.
- Note that the given step can be similar to multiple steps, return steps with high confidence.

UI Instruction

Placeholder for the executed UI instruction.

Output Guidelines

Please return the output in the JSON format:

- STEP_IDS: The searched step IDs.

Fig. 5. Role-Play Prompt for Oracle Retrieval

4.1 Experimental Setup

To better understand the comparative strengths and weaknesses of automated versus manual execution, we employed 40 soap opera tests from the formative study as our experimental dataset.

4.1.1 Real-Time Bug Auto-detection Setup (RQ1). We assess the number of bugs identified by automated testing after executing the soap opera tests and the accuracy rate of detecting valid bugs. Two authors independently reviewed the entire execution process, reproducing each detected bug on Android devices to verify its validity. Disagreements were resolved through discussion until consensus was reached. Both authors have extensive testing experience. Valid bugs were then submitted to the respective projects to collect feedback.

4.1.2 Soap Opera Test Auto-play Setup (RQ2). We evaluate how many tests automated testing can fully complete. Two authors independently observed the execution process to confirm whether the tests were completed. Since auto-play involves both Planner and Player, we conducted a deeper analysis of incomplete executions. Out of 40 soap opera tests, 430 Planner-Player executions occurred. We used a statistical sampling method [35], sampling 203 pairs for evaluation to ensure the error margin is 0.05 at a 95% confidence level. Planner was assessed on accuracy of identifying the next step and performance of sub-steps generation using a Likert scale [25] where 1 is the worst and 5 is the best. For Player, we evaluated accuracy of parsing steps into structured UI instructions and accuracy of element location. Discrepancies were resolved through discussion.

4.1.3 Application-Specific Knowledge on Automated Testing Setup (RQ3). We conducted an ablation study to investigate the impact of application-specific knowledge (steps and oracles in SKG) on both the auto-play and bug detection for soap opera testing. For step knowledge used in Planner, we disabled the step retrieval feature and examined the resulting execution outcomes, along with the number and validity ratio of detected bugs. For oracle knowledge used in Detector, to ensure the fairness, we reused existing results from Planner and Player, rerunning only the Detector with its oracle retrieval disabled. We then analyzed the new number of detected bugs.

4.1.4 Generalizability of Automated Testing Setup (RQ4). To further validate the generality of our automated testing approach, we conducted soap opera testing on four additional applications: [DuckDuckGo](#), [Markor](#), [NewPipe](#), and [MaterialFiles](#). Each application underwent 10 soap opera tests, for a total of 40 tests. We then evaluated the bug detection accuracy and the number of tests that were fully completed by automated soap opera testing.

4.2 Effectiveness in Real-Time Bug Auto-detection (RQ1)

As illustrated in Table 1, automated testing has shown promising capabilities in real-time bug auto-detection. 32, 21, 15 and 23 bugs were confirmed by two authors for Firefox, WordPress, AntennaPod and Amaze respectively. Due to time and resource limitations, we selected 15, 9, 10 and 10 bugs for submission to their respective projects, prioritizing defects over enhancements. For enhancements, we focused on those that significantly impacted user experience. Each report was meticulously crafted with detailed descriptions and accompanied by screenshots or videos for reference, ensuring quality over quantity. After reporting the 15+9+10+10 bugs, 3 bugs have been fixed, 26 confirmed, 2 assigned priority and severity, 2

Table 1. Comparison of Detected Bug Numbers

Approach	TP #	FP #	Acc.
Firefox (Manual ¹)	27	0	1.000
Firefox (Automated)	32	30	0.516
WordPress (Manual ¹)	14	0	1.000
WordPress (Automated)	21	11	0.656
AntennaPod (Manual ¹)	16	0	1.000
AntennaPod (Automated)	15	11	0.577
Amaze (Manual ¹)	23	0	1.000
Amaze (Automated)	23	15	0.605

¹Manual Soap Opera Testing from Formative Study

reopen, 3 marked as duplicates. For example, automated testing identified an unexpected [Issue7365](#) (marking the episode as unplayed in ‘Manage downloads’ does not update the contrast to its unplayed status) during the execution, which has been fixed. The detected bugs span various bug types, as shown in Table 2. Manual testing identified 80 bugs with 100% accuracy, comprising 32.5% UI issues, 55.0% functionality errors, 5.0% performance issues, and 7.5% crashes. Automated testing detected 158 bugs, including 58.2% UI issues (57.6% accuracy) and 38.0% functionality errors (53.3% accuracy), underscoring the need for improved detection accuracy. Besides, automated testing detected fewer crash bugs (2 vs. 6 in manual testing), highlighting the need to enhance bug-triggering capabilities. Due to the advanced natural language and image understanding required for automated testing, we employed multi-modal LLMs (e.g., [GPT-4o](#)), which costs approximately \$1.76 per soap opera test and \$0.77 per true positive bug detected. Automated testing has also exposed areas that urgently need improvement: enhancing the capability to exploration boundary and reducing the rate of incorrectly identified bugs (false positives).

4.2.1 Under-Explored Scenario Boundary.

Looking solely at the number of true positive bugs detected in Table 1, automated testing appears to yield comparable results to manual soap opera testing conducted by participants. However, by delving deeper into the bugs detected, we discovered that automated testing and manual testing tend to identify different bugs. Among all the bugs identified by both automated testing and manual testing, only six were identical. This shows the effectiveness of soap opera testing in finding diverse bugs, but also reveals the varying capabilities of manual testing versus automated testing. Automated testing demonstrates greater sensitivity to enhancements, focusing on aspects such as user usability and design consistency, whereas participants tend to focus more on detecting defects, such as functionality, crashes. Notably, a significant portion of the true-positive bugs identified by automated testing were enhancements: 19 out of 32 for Firefox, 15 out of 21 for WordPress, 11 out of 15 for AntennaPod and 14 out of 23 for Amaze. In comparison, participants in the formative study identified fewer enhancement bugs: 3 out of 27 for Firefox, 1 out of 14 for WordPress, 2 out of 16 for AntennaPod and 1 out of 23 for Amaze. An in-depth comparison of automated and manual soap opera testing revealed that automated testing requires improvement in expanding boundary exploration through creative thinking. This is crucial, as some defects can only be discovered by exploring beyond the initial test scenarios.

As detailed in Section 2.3.2, during the initial soap opera test derived from [Bug1816146](#) on editing a saved login, $P1_{Firefox}$ identified two defect bugs by expanding the boundaries of exploration. Upon comparison, automated testing generates mostly conventional inputs, using typical examples such as usernames and passwords: ‘your_username’ and ‘password’. This input generation might not effectively simulate real-world user behaviors or thoroughly test the boundaries of the system’s error handling capabilities. Thus, automated testing highlighted several enhancements during this testing. On ‘Logins and passwords’ page, automated testing identified an inconsistency where the ‘Exceptions’ button features an icon, diverging from other sections that use text only ([Bug1912617](#)). On ‘Saved passwords’ page, it suggested that the full text of saved password entries should be either visible or accessible through text wrapping or a tooltip, enhancing accessibility ([Bug1912621](#)).

Table 2. Comparison of Detected Bug Types

Bug Type	Manual Testing		Automated Testing	
	# Bug (Pct. ¹)	Acc. ²	# Bug (Pct. ¹)	Acc. ²
UI	26 (32.5%)	100.0%	92 (58.2%)	57.6%
Functionality	44 (55.0%)	100.0%	60 (38.0%)	53.3%
Performance	4 (5.0%)	100.0%	4 (2.5%)	100.0%
Crash	6 (7.5%)	100.0%	2 (1.3%)	100.0%
Overall	80 (100.0%)	100.0%	158 (100.0%)	57.6%

¹Pct.: Percentage of bugs relative to total ²Acc.: True positive bug accuracy

On ‘Add password’ page, automated testing discovered that the URL validation message remains static, failing to update and provide feedback based on the user’s input (Bug1907851). Additionally, the ‘Password’ field on the same page lacks a ‘Show/Hide’ toggle, a feature expected for user convenience (Bug1912202). Inspired by Bug1912202, an author found that ‘Site’ field misses ‘Clear’ button for removing invalid websites (Bug1912200). After reporting these bugs, all were confirmed. Input exploration can be enhanced by developing an input generation module that leverages the GUI state and testing techniques like boundary value analysis to generate diverse inputs, thereby expanding the input exploration boundaries.

4.2.2 Incorrectly Identified Bugs. From Table 1, it is evident that automated testing exhibits a relatively high false positive rate in bug detection. To understand the underlying reasons for this high rate, we conducted a detailed analysis of all 67 incorrectly reported bugs and identified four main causes: LLMs hallucination, screenshot-based GUI presentation, error GUI element location, and unreasonable or unnecessary enhancement suggestions.

LLMs Hallucination (20 out of 67: 29.9%). Of the 67 incorrectly reported bugs, 20 were attributed to hallucinations from LLMs, which break down into two categories: hallucinations about UI design issues (14/20) and unexecuted execution plans (6/20). These hallucinations can arise either directly from the LLMs themselves or from the application of oracle knowledge.

False UI Design Bugs (14 out of 20): Hallucinations in this category include issues such as overlap, obscuring, and GUI design inconsistency. For instance, the LLMs might creatively generate an oracle like “Check if the save icon is enabled even when required fields are empty”, and then erroneously hallucinates a non-existent bug: the save icon is reported as enabled while, in fact, it is disabled. Hallucinations can also be prompted by oracle knowledge from the scenario knowledge base. For example, when the user taps on the three-dot menu in the top right corner, an oracle is retrieved for bug detection stating, “Reduce the height of the 3-dot menu by consolidating some items to make it fit within the screen without requiring scrolling”. Influenced by this oracle, the LLM mistakenly perceives that “The menu appears to extend beyond the screen’s height, which might result in a need for scrolling, especially on smaller screens”, even though this issue does not actually exist.

Unexecuted Execution Plans (6 out of 20): These hallucinations occur when the LLM predicts bugs from actions or sequences that were never executed during testing. Hallucinations influenced by oracle knowledge are particularly intriguing. For instance, an oracle in WordPress states, “The ‘No internet’ snackbar should appear when the bottom loader view becomes visible”. Due to this mention of “no internet”, Detector mistakenly generates a bug report without executing the last two steps described in the S2Rs: “Open Stats section”, “Scroll to map view with network”, “Switch to offline mode”, and “Observe response and rendering”. The mention of “no internet” prompted an unexecuted operation related to switching to offline mode, causing a false positive. However, this oracle can be used strategically to guide LLMs in generating new test branches, expanding the testing scope. By executing steps generated from this oracle, we can assess whether the current scenario reveals issues when the system operates offline. This highlights how oracle knowledge can be leveraged to create new branches, allowing for more comprehensive testing of system behavior. This is particularly effective with oracles that include actionable steps, such as “The vertical search menu/context menu should be dismissed by tapping outside or using the back button”.

Screenshot-based GUI Presentation (12 out of 67: 17.9%). Misreporting can be further divided into two situations: the first occurs due to incomplete rendering/loading at the time of the screenshot, while the second arises from a failure to capture dynamic changes. For instance, a false positive bug from WordPress reported, “Post creation screen does not display any text input fields or content creation tools”, due to incomplete rendering/loading. To address this issue, we can invoke adaptive screenshotting tools like AdaT [12], which dynamically adjusts the screenshotting timing

based on the GUI rendering state to reduce false positives. An example of a false positive caused by failure to capture dynamic changes is “Episodes list does not update after tapping ‘Refresh’”. In reality, the loading animation appeared after tapping “Refresh”, but was not visible in the screenshot due to the screenshooting timing. To address this, video-based bug detection approach or mitigating the issue by taking more frequent and dense screenshots can be adopted.

Error GUI Element Location (7 out of 67: 10.4%). Another type of false bug report is caused by GUI element location errors. For instance, the report “Security information panel does not display after tapping the lock icon” was actually due to the lock icon’s incorrect coordinates, which caused the icon to not be clicked properly. To address this, we need to improve the accuracy of element positioning, potentially by fine-tuning the LLMs for better spatial understanding and interaction.

Unreasonable or Unnecessary Improvement Suggestions (28 out of 67: 41.8%). This type of false bug report arises from generating improvement suggestions that are either unreasonable or unnecessary for the system’s actual functionality. Such suggestions often contribute little or are based on misunderstandings. For example, the suggestion, “The ‘Sponsored’ label for Amazon and Hotels.com is inconsistent with other site labels”, is unreasonable since only these two websites are sponsors, making the call for additional labels unreasonable. Similarly, the suggestion “Lack of sorting or filtering options for the subscriptions page” was unnecessary, as these options are already accessible via the three-dot icon on the page. These suggestions generally result from a lack of understanding of the app’s layout and business context. To address this, we plan to develop a multimodal KG that integrates GUI information as a reference. Additionally, the KG should incorporate a broader range of information sources, such as requirement manuals, design documentation, and other business-relevant resources to provide more comprehensive context.

4.3 Effectiveness in Soap Opera Test Auto-play (RQ2)

We found that 7 out of 10 soap opera tests were fully executed for Firefox, and 6 out of 10 for WordPress, AntennaPod and Amaze. Planner achieved 92.1% accuracy in identifying the next step, with sub-step generation receiving positive feedback, 73.4% rated ‘5’ (best) on a Likert scale, 7.4% rated ‘4’, 6.4% rated ‘3’, 4.4% rated ‘2’ and 8.4% rated ‘1’ (worst). Player had an 84.2% accuracy in parsing steps into structured UI instructions and a 72.9% accuracy in element location. While the Planner performed well, improvements are needed in the Player’s element location accuracy to enhance overall test execution.

In the evaluation, we found that Planner demonstrated the ability to dynamically adjust execution plans in real time, based on the specific state of the GUI. This adaptability allows the Planner to respond effectively to changes in the interface, enhancing its performance in guiding the soap opera test auto-play and even expanding the boundaries of exploration. As shown in Figure 1, after closing a tab, the test requires reopening a closed tab. During execution, the GUI briefly displays a “Tab closed UNDO” banner after the tab is closed. Planner adapts by deciding to use the ‘UNDO’ feature after detecting the tab closure, making it easier to reopen the recently closed tab. However, since the banner only appeared for a short time, by the time the Player attempted to execute the action, the banner had vanished, preventing the operation from completing. After the result analysis, observing this branch exploration inspired the author to discover two new bugs: [Bug1912742](#) (The recently closed tab reappears in its previous position but without the content preview when using the ‘UNDO’ button in the tabs tray) and [Bug1912747](#) (The ‘UNDO’ action highlights the last two tabs instead of just the last one on the tabs tray). Both bugs have been confirmed. This unveils the opportunity for human-AI collaboration for soap opera testing.

Although the Player demonstrated suboptimal performance in element location accuracy, we unexpectedly found that the inaccurate positioning contributed to the exploration of random branches. For instance, while executing the last step in the soap opera test shown in Figure 1,

after selecting a tab, the Player intended to click the Three-dot menu but mistakenly positioned the click on the Share button. This opens the share page and unintentionally explores a random branch. Thanks to the adaptive capabilities of the Planner, it recognized that the Share operation was unrelated to the current test. Then, Planner generated a plan to close the Share page, effectively returning to the main task. However, this detour into the random branch led to the discovery of an unexpected bug: “The back icon turns black and the title reverts to ‘Recently Closed Tabs’ while still displaying the selected websites after canceling a share action on the ‘Recently Closed Tabs’ page”. This bug was identified due to the random branch exploration and was reported and fixed as [Bug1912905](#). After analyzing the bug discovery process, inspired by this branch exploration, the author applied equivalence class to test similar branches in other pages with selection and share functionality. As a result, two similar bugs were found on the Tabs Tray page and Bookmarks page, reported as [Bug1912910](#) (confirmed) and [Bug1912912](#) (assigned priority and severity).

This example highlights two key points. First, Planner can ensure that even when exploring random branches, it can still return to and complete the main task. Second, Player’s inaccurate element positioning shows how unintentional actions can lead to the discovery of new issues. In future iterations of automated soap opera testing, introducing a certain degree of randomness could help expand exploration boundaries and trigger unexpected bugs. Besides, inspired by the exploratory execution of automated testing, the authors identified 17 additional bugs, of which 3 were fixed, 9 confirmed, 2 assigned priority and severity, 1 reopened, 1 marked as a duplicate, and 1 marked as wontfix.

4.4 Usefulness of Application-Specific Knowledge (RQ3)

As shown in Table 3, application-specific knowledge is crucial for improving the effectiveness and efficiency of soap opera testing. While testing can proceed without it, its absence diminishes performance. For newer or less popular applications, the SKG can be built alongside automated exploration. As domain knowledge accumulates, the refined SKG can guide subsequent soap opera tests, enhancing the overall testing process.

Table 3. Ablation Study on Bug Detection Accuracy

Tool	Firefox			WordPress			AntennaPod			Amaze		
	TP #	FP #	Acc.	TP #	FP #	Acc.	TP #	FP #	Acc.	TP #	FP #	Acc.
Automated Testing	32	30	0.516	21	11	0.656	15	11	0.577	23	15	0.605
~ w/o Oracle Knowledge	8	29	0.216	8	11	0.421	9	19	0.321	13	19	0.406
~ w/o Step Knowledge	10	23	0.303	7	35	0.167	13	26	0.333	4	17	0.190

Without the support of oracle knowledge, the number of effectively identified true-positive bugs significantly decreases, and the proportion of true bugs drops noticeably, specifically, from 0.516 to 0.216 for Firefox, 0.656 to 0.421 for WordPress, 0.577 to 0.321 for AntennaPod and 0.605 to 0.406 for Amaze. This highlights the critical role that oracle knowledge plays in enhancing real-time bug auto-detection. Even so, the oracle knowledge from SKG still has room for improvement. First, oracles with similar meanings often appear repeatedly. For example, as shown in Figure 1, after executing “Click on the tabs tray,” the retrieved oracles include “Tooltip should display explanatory text” and “View the tooltip with missing text,” which are essentially duplicates. These could be merged using semantic clustering to avoid redundancy. Second, some oracles are irrelevant to the current testing scenario. In the same example, one of the retrieved oracles is “The thumbnail image for the image opened in a new tab should be correctly displayed,” which is irrelevant because no new tab has been opened in the current soap opera test. However, these oracles are beneficial for generating new branch to expand the exploratory boundary. Lastly, for infeasible oracles, such as “Locate the tab switcher icon inside the tab tray” (where the current GUI no longer uses this design), filtering mechanisms should be implemented to remove these infeasible oracles, which could help reduce hallucinations and improve the accuracy of bug detection.

Without the support of step knowledge, the number of fully executed soap opera tests dropped significantly, from 7 to 3 for Firefox, from 6 to 3 for WordPress, from 6 to 5 for AntennaPod and from 6 to 1 for Amaze. As shown in Table 3, this also significantly reduced the effectiveness of real-time bug auto-detection, with detection rates dropping from 0.516 to 0.303 for Firefox, 0.656 to 0.167 for WordPress, 0.577 to 0.333 for AntennaPod and 0.605 to 0.190 for Amaze. In the analysis of step knowledge, we found that the granularity and breadth of the steps have a significant impact on the quality of automated soap opera testing. For example, in Firefox's knowledge base, the scenario sources are extensive and come from a large user base, including developers, testers, and regular users, all of whom have varying writing habits. The advantage of this diversity is that while some scenarios are written with poor quality, often missing UI instructions, other contributors' writings help fill in these gaps. However, for an app like AntennaPod, which has fewer users writing issues and pull requests, as well as a smaller development team, the missing UI instructions are not as readily supplemented. In contrast, for WordPress, the scenarios often come from pull requests containing high-quality, fine-grained steps written by developers. This is beneficial for auto-play, as it provides clear, detailed instructions, but the downside is that some features are not covered, making it inaccessible and limiting the effective exploration of certain features. For example, as seen in Table 3, the omission of step knowledge in WordPress led to the most significant drop in TP and a substantial increase in FP. To address these issues and further tackle the cold-start problem, a UI auto-exploration model could be designed to automatically construct a comprehensive KG of the app's functionality and UI operations.

4.5 Generalizability of Automated Soap Opera Testing (RQ4)

The average steps per soap opera test were 4.7 (DuckDuckGo), 4.3 (Markor), 4.4 (NewPipe), and 3.4 (MaterialFiles). Full execution occurred in 5/10 tests for DuckDuckGo, Markor, and MaterialFiles, and in 6/10 for NewPipe. Bug detection accuracies were 0.500, 0.706, 0.526, and 0.455 for DuckDuckGo, Markor, NewPipe, and MaterialFiles, respectively. These results further demonstrate the generalizability of our approach. Additionally, our observations indicate that similar functionalities across applications tend to yield analogous bugs, as supported by previous work [24, 40]. For instance, DuckDuckGo exhibits the same "Find in page" issue as Firefox, evidenced by DuckDuckGo's [Issue5636](#) and [Issue5637](#), along with Firefox's [Bug1913291](#) and [Bug1913295](#). A similar issue also exists in Google Chrome browser, as reported in [Issue400119366](#). This demonstrates that certain knowledge, like steps and oracles, can be shared across applications to provide broader guidance. However, differences in UI structure, naming, or operational steps may introduce noise and compromise precision. Investigating a shared scenario KG that balances these benefits with application-specific accuracy is a promising direction.

5 Road Ahead for Automated Soap Opera Testing

Based on the aforementioned research, we found that automated soap opera testing directed by LLMs and scenario knowledge shows promising performance. However, several issues have also emerged that need to be addressed. In this section, we summarize the observations and outline the road ahead for automated soap opera testing.

5.1 The Synergy of Neural and Symbolic Approaches

Based on the experimental results, effective soap opera testing with LLMs requires comprehensive application-specific knowledge. Neural models provide essential NLU and vision capabilities, enabling the system to interpret diverse UI states and instructions. However, without application-specific symbolic knowledge, such as precise steps and oracles, neural models lack the contextual grounding needed to handle complex, domain-specific scenarios effectively. Integrating symbolic

knowledge equips the system with structured and detailed application-specific information. This synergy allows neural models to offer adaptability and generalization, while symbolic elements ensure precision and domain relevance. Thus, combining neural and symbolic approaches is essential.

5.1.1 KG Construction via Automated Exploration and Cross-App Knowledge. Even though crowdsourced information (e.g., bug reports, user manuals) provides a valuable resource, it often faces limitations in scale and detail, resulting in challenges such as: a) abstract step descriptions, making direct execution difficult; b) limited knowledge, leading to incomplete coverage of certain software functionalities. This issue is particularly significant in newer software projects that lack sufficient crowdsourced contributions. As a solution, generation of functionality KG of UI instructions by automated software exploration [42, 44, 49] offers a promising solution. Alternatively, shared knowledge across applications can provide broader guidance [24, 40, 48], though it may introduce noise that affects precision. Exploring the construction of shared Scenario KG that balance the benefits of shared knowledge with app-specific precision is another promising direction.

5.1.2 Expanding Exploration Boundaries with Diverse Knowledge. When LLMs perform soap opera testing, their ability to expand the exploration boundary through creative thinking is still basic. This limitation leads to many bugs not being triggered. Therefore, how to assist LLMs in generating exploration boundaries using diverse knowledge is a direction that requires further exploration. Exploration boundaries can be divided into two types: input exploration and branch exploration. For input exploration, we can structure the input field as a separate node when constructing the KG. The KG can integrate previously existing input values from various crowdsourced sources, such as bug reports. Further, by combining software testing techniques with the LLMs' GUI understanding capabilities, various representative inputs can be generated based on the input context. This approach enables effective input boundary exploration, ensuring a comprehensive evaluation of possible input scenarios. For branch exploration, we can enhance the KG by abstracting scenarios and integrating similar ones at a higher level, allowing LLMs to perform equivalence class reasoning and logically explore related branches. Additionally, oracles can be used to guide LLMs in generating effective branches. Random branch exploration (e.g., accidentally tap the wrong button) may also yield unexpected results, so during soap opera test execution, we can introduce random factors via LLMs to facilitate random exploration. By utilizing GraphRAG, we can better leverage the structured information within the KG to enhance this process. It is important to ensure that boundary exploration is conducted while completing the main tasks, to avoid getting lost in creative exploration. However, determining the most effective balance between creativity and precision is still an unresolved issue. Therefore, exploring the creativity and precision trade-off and identifying the optimal balance is a very important and interesting research direction.

5.1.3 Video-Based Bug Detection for GUI Interactions. By capturing and analyzing video streams of GUI interactions, we can identify bugs missed by static snapshots or logs, particularly issues with transitions, animations, and dynamic elements. A key challenge is processing large volumes of video data while synchronizing it with UI instructions. Overlaying UI instructions as subtitles in the video can improve clarity and help identify anomalies.

5.2 Human-AI Co-learning

We found that not only can humans provide insights for AI in automated soap opera testing, but AI's behavior can also inspire human creativity, leading to the discovery of more bugs. Inspired by automated soap opera testing, the authors identified 17 additional bugs, of which 3 were fixed, 9 confirmed, 2 assigned priority and severity, 1 reopened, 1 marked as a duplicate, and 1 marked as wontfix. Additionally, we observe some false positive situations caused by LLM hallucinations are a

challenge for LLMs, but humans can quickly distinguish between true and false bugs. For instance, a false bug such as “The ‘Done’ button appears slightly larger and misaligned...” is easily identifiable as incorrect by a human tester. Given the above two findings, as we transition from manual to automated soap opera testing, we could adopt a middle-ground approach by designing a Human-AI collaborative intelligence method to assist the process. On one hand, this can reduce the incorrectly identified bugs caused by LLM hallucinations, and on the other hand, human involvement may also be inspired by the automated execution process, leading to the discovery of even more bugs. The challenge with this approach lies in when and how humans should intervene and in which way human and AI can effectively collaborate. For example, we can combine program analysis, testing and LLM-based inference to generate a confidence score for identified bugs, and when the score falls below a certain threshold, human intervention would be triggered.

5.3 Soap Opera Testing Integrated with Broader Software Engineering Knowledge

Instead of stopping after discovering a single bug, soap opera testing excels at continuing to explore around the discovered bug, uncovering various related issues, and even identifying different manifestations of the same root cause. For example, in [Issue7349](#) and [Issue7350](#), one involved infinite loading, and the other a crash. After reporting both, the developer discovered they stemmed from the same underlying issue. To improve this process, we propose integrating soap opera testing with broader software engineering knowledge, such as code, by constructing a scenario-code linked KG. This KG would link scenarios to code, helping identify shared root causes when multiple bugs point to the same code segment. This approach enables concentrated bug fixing, addressing root causes instead of individual bugs. Additionally, we can leverage the different scenarios linked to the same code segment to recommend branches for further exploration. The challenge in this research direction lies in how to accurately and finely restore the traceability between scenarios and code.

6 Threats to Validity

To mitigate potential human bias due to the small number of participants in the formative study, we recruited professionals with at least two years of experience in soap opera testing of large interactive systems with GUIs, such as desktop or mobile applications. To further reduce bias during the evaluation, two authors independently assessed the results, discussing disagreements to reach consensus. Additionally, we reported the bugs identified in both the formative study and automated soap opera testing to the respective open-source repositories for developer confirmation.

A significant external validity threat is that the evaluation was conducted on limited applications. However, these applications span different categories. The diversity of their functionalities helps mitigate potential bias. We plan to deploy automated soap opera testing on a broader range of software systems to further validate its feasibility. The KG construction relies on crowdsourced information. For newer or less popular software that lacks such data, our approach remains functional even without SKG, albeit with limited effectiveness. As our approach iteratively explores, tests, and records the process, the SKG can be gradually constructed, mitigating the cold-start problem through the continuous accumulation of knowledge during the testing process. Alternatively, we aim to develop an automated software exploration method to generate a functionality KG of UI instructions to guide automated soap opera testing.

7 Related Work

Manual exploratory testing has been shown to be effective at the system testing level [1, 2, 4, 10, 18, 22, 28, 41, 46]. Principles and guidelines for conducting and managing ET have been proposed [2, 8, 9, 28, 45]. Tapir [7] generates navigational test cases to minimize duplication but requires manual execution. Scenario-based exploratory testing, also known as soap opera testing [8], involves

designing complex scenarios to provoke failures. Recent work [36–38] has supported soap opera testing by constructing a system KG (SYSKG) of user tasks and failures and generating soap opera tests by combining relevant scenarios based on the SYSKG. SoapOperaTG [37] is developed to optimize the usage of soap opera tests. Experimental results [36–38] show that these generated soap opera tests are highly effective in identifying bugs. However, the manual execution required has hindered the large-scale adoption. Our work is the first to investigate the feasibility, challenges and road ahead of automated soap opera testing directed by LLMs and scenario knowledge. Yu et al. [48] propose SCENTEST, a novel scenario-based approach for mobile app testing that leverages an event KG and GUI image understanding. While SCENTEST focuses on detecting crash bugs, our approach identifies a broader range of unexpected behaviors, including UI issues, functionality errors, performance problems, and crashes. Moreover, soap opera testing emphasizes creative exploration of diverse inputs and side tasks, uncovering bugs beyond predefined steps. This dynamic and adaptive nature extends beyond SCENTEST’s traditional scenario-based testing scope.

Given the remarkable performance of LLMs in NLP and vision, researchers have begun to explore their applicability or impact on software engineering, such as automated code completion [20, 30], software development [16, 33], security vulnerabilities [14, 31], software exploration [42, 44, 49] and software testing [26, 27, 39]. However, no work has yet explored the use of LLMs to automate soap opera testing. Maryam et al. [39] developed AXNav, a multi-agent system leveraging multi-modal LLMs to interpret accessibility tests in NL and replay the instructions on mobile devices while manipulating relevant accessibility features. AXNav currently detects issues related to four accessibility features: VoiceOver, Dynamic Type, Bold Text, and Button Shapes. Unlike AXNav, which focuses on replaying accessibility tests for specific accessibility issues, our work explores automated soap opera testing, which is not confined to specific bug types. It enables the detection of any bugs, even those beyond predefined oracles, during the execution of soap opera tests. Furthermore, it allows to vary the tests as appropriate, enhancing both the breadth and depth of the testing process.

Recent work has investigated reproducing bugs from the S2Rs in bug reports and issues [11, 34, 50] or from bug videos [5, 15]. Su et al. [8, 36, 37] have extracted useful sections, such as S2Rs and EBs, to generate soap opera tests. BUGINE [24, 40] recommends bugs from certain apps to developers of similar apps for testing. None of these approaches leverage the abundant scenario knowledge present in bug reports, issues, and pull requests to guide automated soap opera testing.

8 Conclusions

This paper identifies key insights for effective manual soap opera testing and the challenges in automating this process through a formative study. Based on these insights and challenges, we develop a multi-agent system that leverages LLMs and a scenario knowledge graph to automate soap opera testing. The system comprises three multi-modal agents, Planner, Player, and Detector, which collaborate to execute soap opera tests and identify potential bugs. Experimental results demonstrate the feasibility of automated soap opera testing, with identified bugs being confirmed and fixed by the development team. However, there remains a gap in achieving the effectiveness of manual soap opera testing, particularly in addressing under-explored scenario boundaries and reducing incorrectly identified bugs (false positives). We further discuss opportunities, challenges and potential solutions in the road ahead for automated soap opera testing, focusing on three key areas: the synergy of neural and symbolic approaches, human-AI co-learning, and integrating soap opera testing with broader software engineering practices.

Data Availability

Replication package with code and dataset: <https://github.com/SuYanqi/SoapOperaTesting>

References

- [1] Wasif Afzal, Ahmad Nauman Ghazi, Juha Itkonen, Richard Torkar, Anneliese Andrews, and Khurram Bhatti. 2015. An experiment on the effectiveness and efficiency of exploratory testing. *Empirical Software Engineering* 20 (2015), 844–878. doi:10.1007/s10664-014-9301-4
- [2] Jonathan Bach. 2000. Session-based test management. *Software Testing and Quality Engineering Magazine* 2, 6 (2000), 32–37.
- [3] James Bach. 2003. Exploratory testing explained. Online: <http://www.satisfice.com/articles/et-article.pdf> (2003), 1–10.
- [4] James Bach. 2004. Exploratory testing. *The testing practitioner* (2004), 253–265.
- [5] Carlos Bernal-Cárdenas, Nathan Cooper, Kevin Moran, Oscar Chaparro, Andrian Marcus, and Denys Poshyvanyk. 2020. Translating video recordings of mobile app usages into replayable scenarios. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 309–321. doi:10.1145/3377811.3380328
- [6] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [7] Miroslav Bures, Karel Frajtak, and Bestoun S Ahmed. 2018. Tapir: Automation support of exploratory testing using model reconstruction of the system under test. *IEEE Transactions on Reliability* 67, 2 (2018), 557–580. doi:10.1109/TR.2018.2799957
- [8] Hans Buwalda. 2004. Soap opera testing. *Better Software* 6, 2 (2004), 30–37.
- [9] JD Cem Kaner. 2013. An introduction to scenario testing. *Florida Institute of Technology, Melbourne* (2013), 1–13.
- [10] JD Cem Kaner and James Bach. 2006. The nature of exploratory testing. (2006).
- [11] Mattia Fazzini, Martin Prammer, Marcelo d’Amorim, and Alessandro Orso. 2018. Automatically translating bug reports into test cases for mobile apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 141–152. doi:10.1145/3213846.3213869
- [12] Sidong Feng, Mulong Xie, and Chunyang Chen. 2023. Efficiency matters: Speeding up automated testing with gui rendering inference. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 906–918. doi:10.1109/ICSE48619.2023.00084
- [13] Google. 2024. Android Debug Bridge (adb). <https://developer.android.com/studio/command-line/adb>.
- [14] Anastasiia Grishina. 2022. Enabling automatic repair of source code vulnerabilities using data-driven methods. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 275–277.
- [15] Madeleine Havranek, Carlos Bernal-Cárdenas, Nathan Cooper, Oscar Chaparro, Denys Poshyvnyak, and Kevin Moran. 2021. V2S: a tool for translating video recordings of mobile app usages into replayable scenarios. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 65–68.
- [16] Saki Imai. 2022. Is github copilot a substitute for human pair-programming? an empirical study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 319–321.
- [17] Juha Itkonen and Mika V Mäntylä. 2014. Are test cases needed? Replicated comparison between exploratory and test-case-based software testing. *Empirical Software Engineering* 19, 2 (2014), 303–342. doi:10.1007/s10664-013-9266-8
- [18] Juha Itkonen, Mika V Mäntylä, and Casper Lassenius. 2012. The role of the tester’s knowledge in exploratory software testing. *IEEE Transactions on Software Engineering* 39, 5 (2012), 707–724. doi:10.1109/TSE.2012.55
- [19] Juha Itkonen and Kristian Rautiainen. 2005. Exploratory testing: a multiple case study. In *2005 International Symposium on Empirical Software Engineering, 2005. IEEE*, 10–pp. doi:10.1109/ISESE.2005.1541817
- [20] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering*. 1219–1231. doi:10.1145/3510003.3510203
- [21] Cem Kaner. 2008. A tutorial in exploratory testing. *Tutorial presented at QUEST2008*. (Available online at: <http://www.kaner.com/pdfs/QAExploring.pdf>, accessed: 26 Jan 2014) (2008).
- [22] Cem Kaner, Jack Falk, and Hung Q Nguyen. 1999. *Testing computer software*. John Wiley & Sons.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [24] Ziqiang Li and Shin Hwei Tan. 2020. Bugine: a bug report recommendation system for Android apps. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*. 278–279.
- [25] Rensis Likert. 1932. A technique for the measurement of attitudes. *Archives of psychology* (1932).
- [26] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Fill in the blank: Context-aware automated text input generation for mobile gui testing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1355–1367 (2023). doi:10.1109/ICSE48619.2023.00119
- [27] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Zhilin Tian, Yuekai Huang, Jun Hu, and Qing Wang. 2024. Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639118

- [28] James Lyndsay and Neil Van Eeden. 2003. Adventures in session-based testing. *Workroom Productions Ltd.* May 27 (2003).
- [29] Torvald Mårtensson, Daniel Ståhl, Antonio Martini, and Jan Bosch. 2021. Efficient and effective exploratory testing of large-scale software systems. *Journal of Systems and Software* 174 (2021), 110890. doi:10.1016/j.jss.2020.110890
- [30] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *Proceedings of the 45th International Conference on Software Engineering (ICSE'23)*. doi:10.1109/ICSE48619.2023.00205
- [31] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2339–2356.
- [32] Dietmar Pfahl, Huishi Yin, Mika V Mäntylä, and Jürgen Münch. 2014. How is exploratory testing used? A state-of-the-practice survey. In *Proceedings of the 8th ACM/IEEE international symposium on empirical software engineering and measurement*. 1–10. doi:10.1145/2652524.2652531
- [33] Advait Sarkar, Andrew D Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence? *arXiv preprint arXiv:2208.06213* (2022).
- [34] Feng Sidong and Chen Chunyang. 2024. Prompting Is All Your Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the 46th International Conference on Software Engineering (ICSE'24)*.
- [35] Ravindra Singh and Naurang Singh Mangat. 2013. *Elements of survey sampling*. Vol. 15. Springer Science & Business Media.
- [36] Yanqi Su, Zheming Han, Zhenchang Xing, Xin Xia, Xiwei Xu, Liming Zhu, and Qinghua Lu. 2022. Constructing a System Knowledge Graph of User Tasks and Failures from Bug Reports to Support Soap Opera Testing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13. doi:10.1145/3551349.3556967
- [37] Yanqi Su, Zheming Han, Zhenchang Xing, Xiwei Xu, Liming Zhu, and Qinghua Lu. 2023. SoapOperaTG: A Tool for System Knowledge Graph Based Soap Opera Test Generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 51–54.
- [38] Yanqi Su, Dianshu Liao, Zhenchang Xing, Qing Huang, Mulong Xie, Qinghua Lu, and Xiwei Xu. 2024. Enhancing exploratory testing by large language model and knowledge graph. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639157
- [39] Maryam Taeb, Amanda Swearngin, Eldon Schoop, Ruijia Cheng, Yue Jiang, and Jeffrey Nichols. 2024. Axnav: Replaying accessibility tests from natural language. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 1–16. doi:10.1145/3613904.3642777
- [40] Shin Hwei Tan and Ziqiang Li. 2020. Collaborative bug finding for android apps. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1335–1347. doi:10.1145/3377811.3380349
- [41] Jarle Våga and Ståle Amland. 2002. Managing high-speed web testing. In *Software quality and software testing in internet times*. Springer, 23–30.
- [42] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling conversational interaction with mobile ui using large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–17.
- [43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [44] Hao Wen, Yuan Chun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2023. Empowering llm to use smartphone for intelligent task automation. *arXiv e-prints* (2023), arXiv–2308.
- [45] James A Whittaker. 2009. *Exploratory software testing: tips, tricks, tours, and techniques to guide test design*. Pearson Education.
- [46] Bill Wood and David James. 2003. Applying session-based testing to medical software. *Medical Device and Diagnostic Industry* 25, 5 (2003), 90–103.
- [47] Yiheng Xiong, Mengqian Xu, Ting Su, Jingling Sun, Jue Wang, He Wen, Geguang Pu, Jifeng He, and Zhendong Su. 2023. An Empirical Study of Functional Bugs in Android Apps. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023)*. 1319–1331. doi:10.1145/3597926.3598138
- [48] Shengcheng Yu, Chunrong Fang, Mingzhe Du, Zimin Ding, Zhenyu Chen, and Zhendong Su. 2024. Practical, Automated Scenario-based Mobile App Testing. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3414672
- [49] Zhizheng Zhang, Xiaoyi Zhang, Wenxuan Xie, and Yan Lu. 2023. Responsible task automation: Empowering large language models as responsible task automators. *arXiv preprint arXiv:2306.01242* (2023).
- [50] Yu Zhao, Tingting Yu, Ting Su, Yang Liu, Wei Zheng, Jingzhi Zhang, and William GJ Halfond. 2019. Recdroid: automatically reproducing android application crashes from bug reports. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 128–139. doi:10.1109/ICSE.2019.00030

Received 2024-09-13; accepted 2025-01-14